

RETAIL SALES FORECASTING: PREDICTING WEEKLY SALES FOR WALMART STORES USING MACHINE LEARNING REGRESSION MODELS

**Project Report Submitted in fulfillment of the
Requirements for the Award of the Internship of
Summer Training Program by IDRA**

Data Science and AI

By:

Name: [PRATHAM KUMAR RANA]

Institute: [NIT JAMSHEDPUR]

Institute Roll No.: [2023UGMM002]

Enrollment No.: IDRA-[2026-323130]

Under the Supervision of

Dr. Shaheena Salam

Through the platform:



India Data Research Academy

DECLARATION

I hereby declare that the project report titled “Retail Sales Forecasting: Predicting Weekly Sales for Walmart Stores Using Machine Learning Regression Models”, submitted in fulfillment of the requirements for the Summer Training Program internship conducted by the India Data Research Academy (IDRA), is my own original work. The data analysis, code, and results presented in this report were carried out by me under the supervision of Dr. Shaheena Salam. All external sources, datasets, and references used in the preparation of this report have been duly acknowledged.

ACKNOWLEDGEMENT

We sincerely express our gratitude to the IDRA Training Program for providing us with this valuable learning opportunity. We are especially thankful to our mentor, Abrar Mir Sir, for sharing his valuable knowledge, guidance, and practical insights throughout the training, which greatly contributed to our learning and the completion of this project.

We would also like to express our sincere gratitude to Dr. Shaheena Salam for supervising our project and providing the necessary support and direction. We are thankful to our institute for providing us with the resources and environment required to carry out this work.

We also acknowledge the dataset and source providers whose resources were used in this project. Finally, we thank everyone who directly supported us during the training and project work.

ABSTRACT

Accurately forecasting weekly retail sales is critical for inventory planning, staffing, and promotional decision-making. This project analyzes historical weekly sales data from 45 Walmart stores to build a regression model that predicts Weekly_Sales using store identity, holiday indicators, and macroeconomic variables (temperature, fuel price, CPI, and unemployment). The dataset, containing 6,435 records with no missing values or duplicate rows, was cleaned, feature-engineered with calendar attributes (month, week, quarter), and explored through univariate, bivariate, and correlation analyses. An independent t-test confirmed that holiday weeks show a statistically significant increase in average sales compared to non-holiday weeks ($t = 2.680$, $p = 0.0076$). Four regression algorithms — Linear Regression, Decision Tree, Random Forest, and Gradient Boosting — were trained and evaluated, with tree-based ensembles substantially outperforming the linear baseline (R^2 of 0.15 versus above 0.90), indicating that the relationship between the predictors and Weekly_Sales is highly non-linear and dominated by store-level differences. Hyperparameter tuning via RandomizedSearchCV and GridSearchCV further improved performance, with the tuned Gradient Boosting model achieving the best result ($RMSE \approx 92,146$, $R^2 \approx 0.974$) on held-out test data. The findings show that store identity is the strongest driver of sales variation, holidays materially lift revenue, and macroeconomic indicators contribute weak individual but meaningful combined predictive signal. The report concludes with practical recommendations for inventory planning, store-specific forecasting strategies, and periodic model retraining.

Keywords: : Retail Sales Forecasting; Regression Analysis; Random Forest; Gradient Boosting; Hyperparameter Tuning

TABLE OF CONTENTS

1. Introduction
 2. Problem Statement and Motivation
 3. Objectives
 4. Research Questions
 5. Dataset Description and Data Understanding
 6. Data Cleaning
 7. Data Preprocessing
 8. Exploratory Data Analysis
 9. Statistical Analysis
 10. Feature Engineering
 11. Model Development
 12. Model Evaluation
 13. Results and Findings
 14. Discussion
 15. Limitations
 16. Conclusion
 17. Recommendations and Future Work
 18. References
- Appendix A. Code
- Appendix B. Additional Figures and Tables

1. INTRODUCTION

1.1 Background

Large multi-store retailers generate vast volumes of transactional data every week, and translating that data into accurate, store-level sales forecasts is central to modern retail operations. Weekly sales at any given store are shaped by a mix of structural factors (store size, location, and customer base), calendar effects (holidays and seasonality), and macroeconomic conditions (fuel prices, inflation, and unemployment). Understanding and modelling these drivers allows a retailer to move from reactive to proactive planning of inventory, staffing, and promotions.

1.2 Context of the Problem

This project uses a historical weekly sales dataset from 45 Walmart stores. Store operations, supply-chain, and finance teams are the primary stakeholders who would use a sales forecasting model: accurate predictions help avoid stock-outs and overstocking, support labor scheduling, and inform promotional timing around holidays. The practical goal is a model that takes store identity, calendar position, and macroeconomic indicators as inputs and outputs a reliable estimate of weekly sales.

1.3 Scope of the Study

The study covers weekly, store-level aggregated sales for 45 stores using the variables available in the provided dataset (Store, Date, Weekly_Sales, Holiday_Flag, Temperature, Fuel_Price, CPI, and Unemployment). It does not cover department-level sales, individual transaction or basket-level data, competitor activity, promotional calendars beyond the holiday flag, or store-level demographic data, none of which were available in the source dataset.

2. PROBLEM STATEMENT AND MOTIVATION

How accurately can a store's weekly sales be predicted using its identity, calendar position, and prevailing macroeconomic conditions? Manual or purely historical-average forecasting does not adapt well to holiday effects or macroeconomic shifts, and can lead to costly inventory and staffing misallocation. A data-driven regression approach that learns the (potentially non-linear) relationships between these predictors and Weekly_Sales can provide more accurate, store-specific forecasts, motivating the modelling work carried out in this project..

3. OBJECTIVES

- To understand and prepare the Walmart weekly sales dataset for analysis.
- To identify important patterns, relationships, and factors (holiday effects, store variation, macroeconomic indicators) affecting weekly sales.
- To engineer calendar-based features and build regression models to predict Weekly_Sales.
- To evaluate candidate models using RMSE, MAE, and R^2 and select the best-performing model.
- To improve the best models through hyperparameter tuning using cross-validation.
- To translate the analytical results into practical recommendations for inventory, staffing, and forecasting strategy.

4. RESEARCH QUESTIONS

1. What are the major characteristics and data-quality properties of the Walmart weekly sales dataset?
2. Which variables are most strongly associated with Weekly_Sales?
3. Does average weekly sales differ significantly between holiday and non-holiday weeks?

4. How much does weekly sales vary across stores, and is store identity a stronger driver than macroeconomic indicators?
5. How accurately can regression models predict weekly sales, and does hyperparameter tuning improve that accuracy?

5. DATASET DESCRIPTION AND DATA UNDERSTANDING

5.1 Data Source

The dataset (P_6_Walmart.csv) was provided as part of the IDRA Summer Training Program LMS materials. It contains historical weekly sales records for 45 Walmart stores along with corresponding holiday flags and regional macroeconomic indicators.

5.2 Dataset Size and Structure

The raw dataset contains 6,435 rows and 8 columns, with no missing values in any column. The target variable is Weekly_Sales, the total sales recorded for a given store in a given week; it was selected as the target because forecasting it is the central business need described in Section 2.

Column 1	DataType	Description
Store	int64	Store identifier (1–45)
Date	object → datetime	Week-ending date (converted to datetime)
Weekly_Sales	float64	Total sales for the store that week (target variable)
Holiday_Flag	int64	1 if the week includes a major holiday, else 0
Temperature	float64	Average regional temperature (°F) for the week
Fuel_Price	float64	Regional fuel price (USD/gallon) for the week
CPI	float64	Consumer Price Index for the region
Unemployment	float64	Regional unemployment rate (%)

Table 1. Summary of the Dataset (6,435 rows × 8 original columns)

Descriptive statistics for the numerical columns (from `df.describe()`) are as follows: Weekly_Sales ranges from about 209,986 to 3,818,686 (mean $\approx$ 1,046,965; std $\approx$ 564,367); Temperature ranges from -2.06°F to 100.14°F; Fuel_Price ranges from 2.47 to 4.47; CPI ranges from 126.06 to 227.23; and Unemployment ranges from 3.88% to 14.31%.

5.3 Data Dictionary

Variable	Meaning	Type	Role
Store	Store identifier	Categorical (int)	Feature
Date	Week-ending date	Datetime	Used to derive calendar features
Weekly_Sales	Total weekly sales (USD)	Numeric	Target
Holiday_Flag	Holiday-week indicator	Binary	Feature
Temperature	Regional average temperature	Numeric	Feature
Fuel_Price	Regional fuel price	Numeric	Feature

CPI	Consumer Price Index	Numeric	Feature
Unemployment	Regional unemployment rate	Numeric	Feature

Table 2. Data Dictionary

6. DATA CLEANING

The raw dataset was found to be largely clean at the row level: it contained no missing values and no duplicate records. The only cleaning actions required were a data-type conversion for the Date column and a review of potential outliers in Weekly_Sales

6.1 Missing Values

`df.isna().sum()` confirmed zero missing values across all eight columns, so no imputation was required.

6.2 Duplicate Records

`df.duplicated().sum()` returned 0, confirming that no duplicate rows were present; the dataset size remained unchanged at 6,435 rows.

6.3 Data Types and Inconsistent Values

The Date column was stored as text in day-month-year format and was converted to a proper datetime type using `pd.to_datetime(df["Date"], format="%d-%m-%Y")`, enabling the extraction of calendar features in Section 10. No other inconsistent categorical values were found; Store (45 unique stores) and Holiday_Flag (binary) were already consistent.

6.4 Outlier Handling

Weekly_Sales was checked for outliers using the IQR method (values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$). This flagged 34 potential outlier rows out of 6,435. On inspection, these values corresponded to high-performing stores and holiday-week sales spikes — genuine, business-meaningful events rather than data errors — so they were retained rather than removed.

6.5 Before vs After Cleaning

Check	Before Cleaning	After Cleaning
Rows	6,435	6,435 (unchanged)
Missing values	0	0
Duplicate rows	0	0
Date column type	object (string)	datetime64

Table 3. Data Quality Before and After Cleaning

7. DATA PREPROCESSING

7.1 Categorical Encoding

Store and Holiday_Flag are already represented as integers. Store was kept as a numeric identifier rather than one-hot encoded, since the tree-based models used in this project (Decision Tree, Random Forest, Gradient Boosting) can split directly on the store id and learn store-specific effects without expanding the feature space into 45 dummy columns. Holiday_Flag is already binary and required no further encoding.

7.2 Feature Scaling and Transformation

Numerical features were standardized with StandardScaler (zero mean, unit variance) only for the Linear Regression model, since distance/coefficient-based linear models are sensitive to feature scale. The scaler was fit

on the training data only and applied to the test data to avoid leakage. Tree-based models (Decision Tree, Random Forest, Gradient Boosting) were trained on the unscaled features, as they are scale-invariant.

7.3 Train-Test Split

The data was split 80/20 into training and test sets using `train_test_split(..., test_size=0.2, random_state=42)`. The split was performed before fitting the scaler, so the scaler's mean and standard deviation were learned only from the training set, preventing information from the test set from leaking into preprocessing.

7.4 Final Feature Set

The final input features used for modelling were: Store, Holiday_Flag, Temperature, Fuel_Price, CPI, Unemployment, Month, Week, and Quarter. The target variable was Weekly_Sales.

8. EXPLORATORY DATA ANALYSIS

8.1 Univariate Analysis

```
weekly_trend = df.groupby("Date")["Weekly_Sales"].sum()
print("\nTotal sales range across all stores/weeks:",
      f"{weekly_trend.min():.0f} TO {weekly_trend.max():.0f}")
```

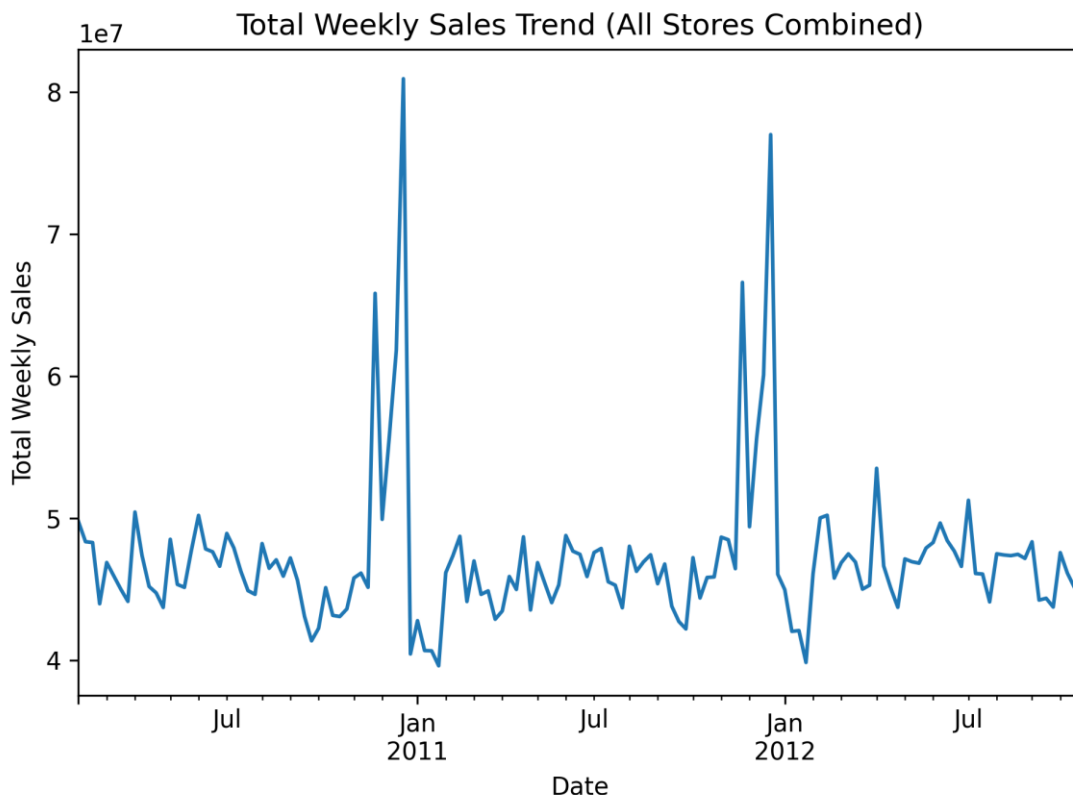


Figure 1. Total weekly sales across Combined

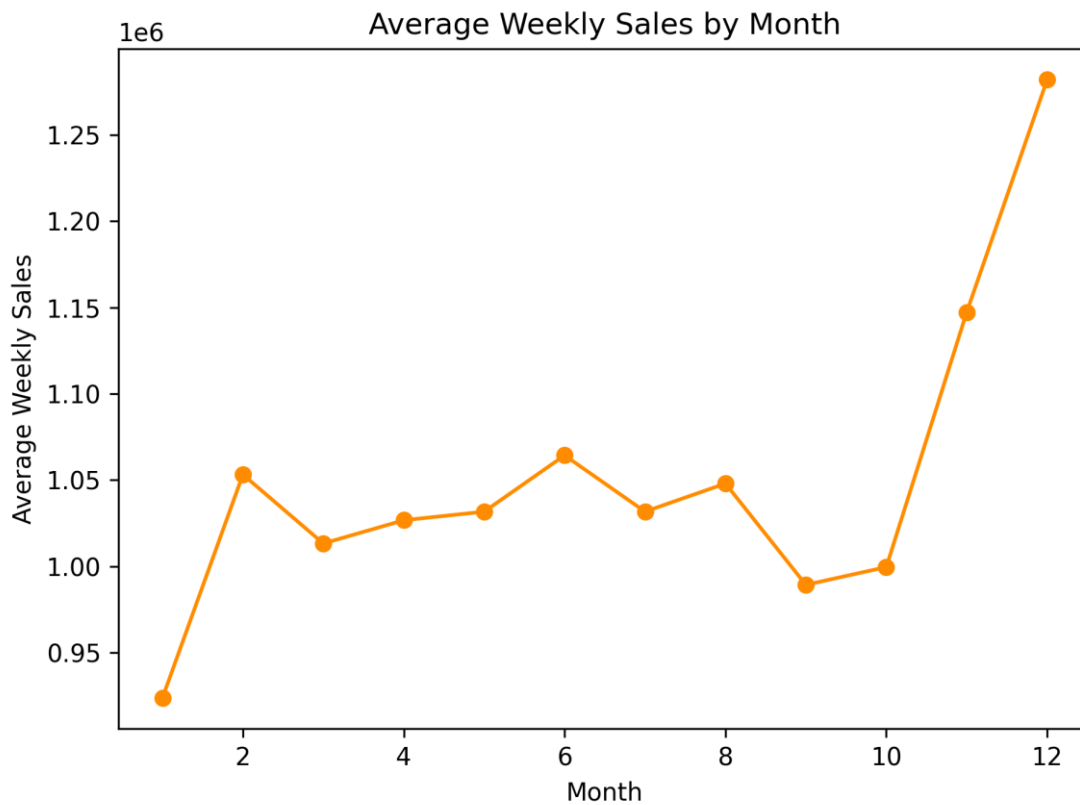


Figure 2. Average Weekly Sales by Month

Weekly_Sales ranges from about 209,986 to 3,818,686, with a mean of roughly 1.05 million and a standard deviation of about 564,000, indicating a wide, right-skewed spread driven mainly by differences between large and small stores rather than by time. The total combined weekly sales across all 45 stores ranged from about 39.6 million to 80.9 million over the observation period.

8.2 Categorical Analysis

Average Weekly_Sales during holiday weeks ($\approx 1,122,888$) is noticeably higher than during non-holiday weeks ($\approx 1,041,256$), an approximate 7.8% uplift. This difference is later confirmed to be statistically significant in Section 9.

```
holiday_avg = df.groupby("Holiday_Flag")["Weekly_Sales"].mean()

non_holiday = holiday_avg[0]
holiday = holiday_avg[1]
```

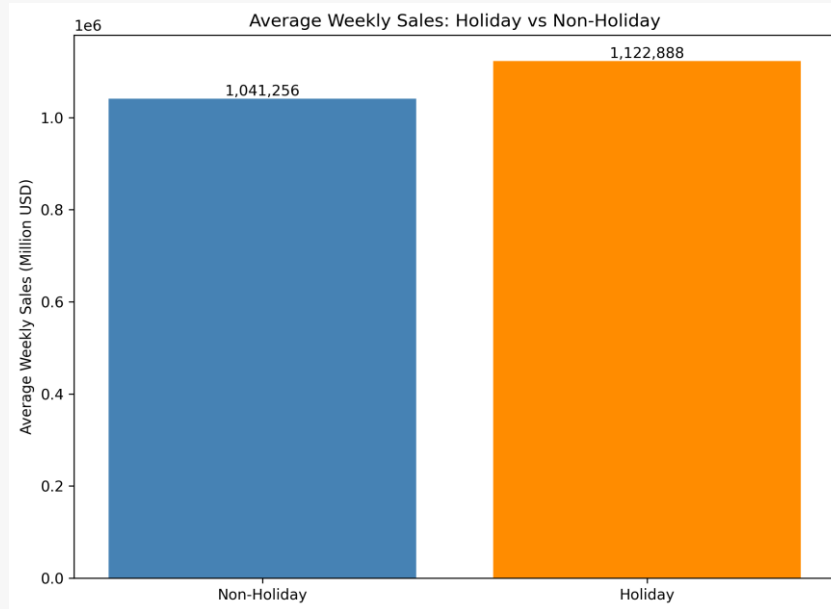


Figure 3. Average Weekly Sales Holiday Vs Non-Holiday Weeks

8.3 Bivariate / Relationship Analysis

```
store_avg_sales = df.groupby("Store")["Weekly_Sales"].mean().sort_values(ascending=False)
top5 = store_avg_sales.nlargest(5) / 1e6
bottom5 = store_avg_sales.nsmallest(5) / 1e6
```

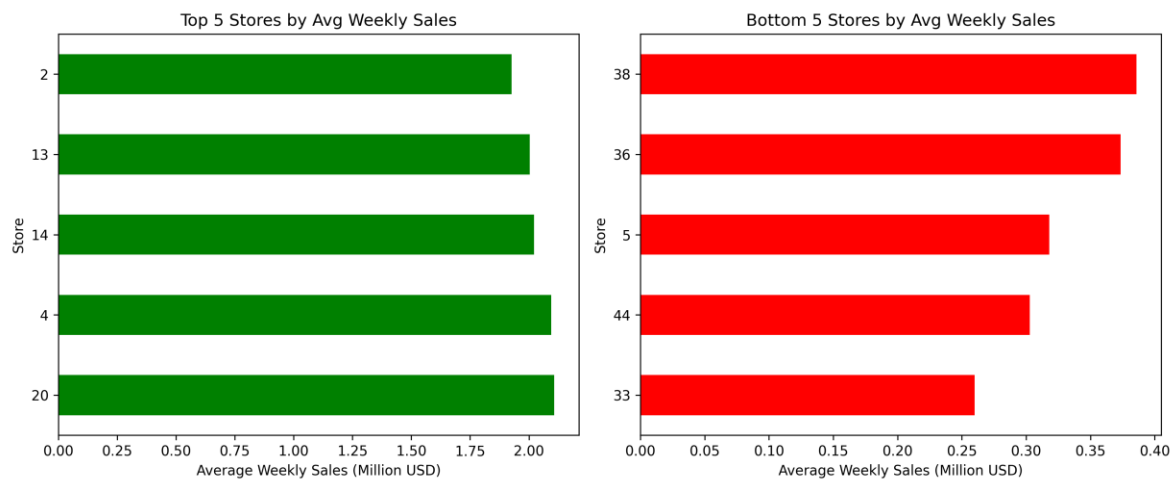


Figure 4. Top 5 and Bottom 5 Stores by Average Weekly Sales

8.4 Correlation Analysis

No individual macroeconomic or calendar feature correlates strongly with `Weekly_Sales` in isolation (all $|r| < 0.11$): Unemployment shows the strongest (negative) relationship at -0.106 , followed by CPI (-0.073) and Temperature (-0.064), while Month, WeekOfYear, and Quarter show small positive associations (around 0.06 – 0.08). This is expected, since weekly sales are dominated by which store it is rather than by week-to-week macroeconomic wobbles. The engineered date parts (Month, WeekOfYear, Quarter, Day) are, however, highly correlated with each other since they are derived from the same Date column, a multicollinearity concern that is manageable for tree-based models but would need addressing (e.g., dropping redundant date parts) if a linear model were the primary choice

```
num_cols =
['Weekly_Sales', 'Holiday_Flag', 'Temperature', 'Fuel_Price', 'CPI', 'Unemployment', 'Year', 'Month',
'WeekOfYear', 'Day', 'Quarter']
corr = df[num_cols].corr()
```

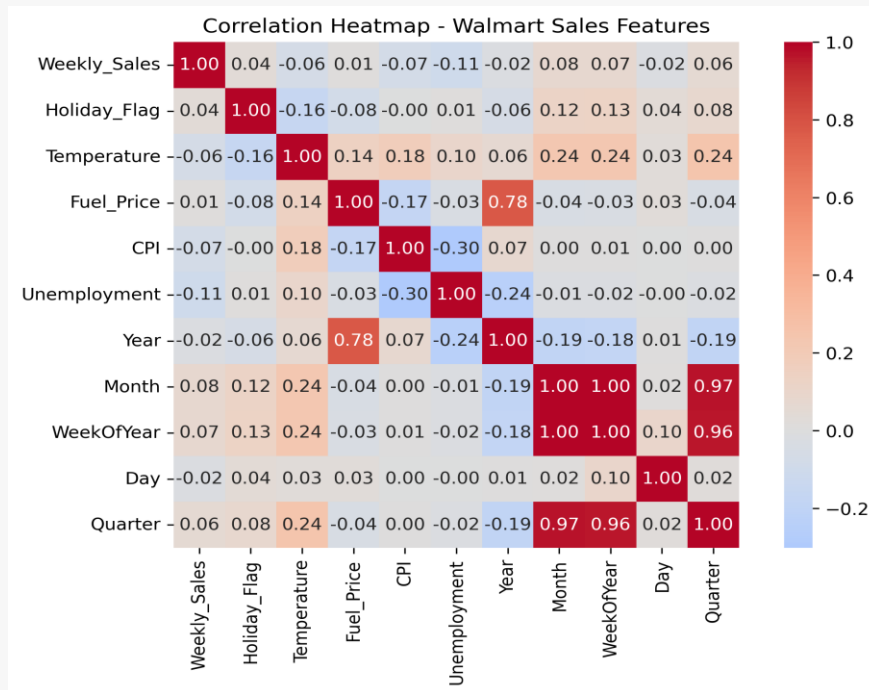


Figure 5. Correlation Heatmap of Numerical Variables

8.5 Other Figures

Additional exploratory visualizations produced in the notebook include the overall weekly sales trend across all stores combined, a boxplot of Weekly_Sales by Holiday_Flag, a boxplot of Weekly_Sales distribution across all 45 stores, and scatter plots of Weekly_Sales against each macroeconomic variable. These are available in the accompanying notebook and can be inserted here as supplementary figures if required.

9. STATISTICAL ANALYSIS

Descriptive statistics for the key numerical variables are summarized below

Statistic	Weekly_Sales	Temperature	Fuel_Price	CPI	Unemployment
Mean	1,046,965	60.66	3.36	171.58	8.00
Std Dev	564,367	18.44	0.46	39.36	1.88
Min	209,986	-2.06	2.47	126.06	3.88
Max	3,818,686	100.14	4.47	227.23	14.31

Table 4. Descriptive Statistics of Key Variables

An independent-samples t-test (Welch's, unequal variance) was used to test whether average Weekly_Sales differs between holiday and non-holiday weeks. The test returned $t = 2.680$, $p = 0.0076$, which is below the conventional 0.05 significance threshold, so the difference in means (holiday weeks higher, as shown in

Figure 2) is statistically significant rather than due to chance. Store-level average sales (Section 8.3) were also compared descriptively across stores; the very large spread between the top and bottom stores indicates that store identity explains far more of the variation in Weekly_Sales than any single macroeconomic indicator, which the correlation results in Section 8.4 corroborate

10. FEATURE ENGINEERING

Since Date alone cannot be used directly by the regression models, several calendar-based features were engineered from it to capture seasonality and time-of-year effects, and the data was then sorted by Store and Date to preserve chronological order within each store.

Engineered Feature	Derived From	Purpose
Year	Date.dt.year	Captures year-level trend
Month	Date.dt.month	Captures month-of-year seasonality
WeekOfYear / Week	Date.dt.isocalendar().week	Captures finer-grained weekly seasonality
Day	Date.dt.day	Captures day-of-month position
Quarter	Date.dt.quarter	Captures quarterly seasonality (used in final feature set)

Table 6. Engineered Features

Of these, Month, Week, and Quarter were retained in the final modelling feature set (Section 7.4) alongside Store, Holiday_Flag, and the macroeconomic variables, since they provide seasonal signal without excessive redundancy with each other.

11. MODEL DEVELOPMENT

11.1 Problem Formulation

This is a supervised regression task: the input features are Store, Holiday_Flag, Temperature, Fuel_Price, CPI, Unemployment, Month, Week, and Quarter, and the continuous target variable is Weekly_Sales. Framing the task as regression (rather than classification) is appropriate because the business need is a numeric sales estimate for planning purposes, not a categorical label.

11.2 Model Selection

Four regression algorithms were trained and compared: Linear Regression as an interpretable baseline; a single Decision Tree Regressor; a Random Forest Regressor (an ensemble of 200 trees); and a Gradient Boosting Regressor. Tree-based ensembles were included because the correlation analysis (Section 8.4) suggested weak individual linear relationships but potentially important non-linear interactions and store-level effects that linear models cannot capture.

11.3 Training the Model

Features and target were split 80/20 into train and test sets. StandardScaler was fit on the training features and applied to Linear Regression only; the tree-based models were trained on the unscaled features

```
X = df[FEATURES]; y = df[TARGET]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(random_state=42, n_estimators=200),
    "Gradient Boosting": GradientBoostingRegressor(random_state=42),
    model.fit(X_train, y_train)
```

Full training and evaluation code is provided in Appendix A.

12. MODEL EVALUATION

12.1 Regression Evaluation

Each model was evaluated on the held-out 20% test set using RMSE (root mean squared error, in the same units as Weekly_Sales), MAE (mean absolute error), and R^2 (proportion of variance explained).

Model	RMSE	MAE	R^2
Linear Regression	522,241.76	433,650.56	0.1534
Decision Tree	130,891.19	75,105.49	0.9468
Random Forest	113,537.68	62,078.42	0.9600
Gradient Boosting	163,769.39	121,585.18	0.9167

Table 7. Regression Model Performance

Linear Regression performs poorly ($R^2 \approx 0.15$), confirming that the relationship between the chosen predictors and Weekly_Sales is highly non-linear and cannot be captured by a linear combination of features. All three tree-based models substantially outperform it, with the untuned Random Forest achieving the best baseline result (RMSE $\approx 113,538$, $R^2 \approx 0.960$).

Hyperparameter tuning was then applied to the two strongest ensemble methods: RandomizedSearchCV (25 candidate combinations, 5-fold cross-validation) for Random Forest, and GridSearchCV (54 combinations, 5-fold cross-validation) for Gradient Boosting

Model	RMSE	R^2	Notes
Random Forest (baseline)	113,537.68	0.9600	n_estimators = 200
Random Forest (tuned)	113,598.52	0.9599	Best params: n_estimators=500, max_depth=None, min_samples_split=2, min_samples_leaf=1, max_features=None
Gradient Boosting (baseline)	163,769.39	0.9167	Default parameters
Gradient Boosting (tuned)	92,146.03	0.9736	Best params: learning_rate=0.1, max_depth=4, n_estimators=300, subsample=1.0

Table 8. Baseline vs Hyperparameter-Tuned Model Performance (Test Set)

Tuning made no meaningful difference for Random Forest (RMSE and R^2 essentially unchanged), suggesting the default configuration was already close to optimal for this feature set. Gradient Boosting, however, improved substantially with tuning, moving from the weakest tree-based baseline ($R^2 \approx 0.917$) to the best-performing model overall (RMSE $\approx 92,146$, $R^2 \approx 0.974$), making tuned Gradient Boosting the recommended model for deployment.

12.2 Classification Evaluation

Not applicable — this project is formulated as a regression problem (Section 11.1), so classification metrics such as confusion matrix, accuracy, precision, recall, and F1-score do not apply.

12.3 Overfitting and Underfitting

All reported metrics are computed on the held-out 20% test set that the models did not see during training, and cross-validation (5-fold) was used during hyperparameter search to guard against overfitting to a single train/test split. Linear Regression shows clear underfitting (very low R^2 on both a linear structure and this data), indicating it lacks the capacity to model the true relationships. The tree-based ensembles show strong, consistent test performance (R^2 above 0.90) without the erratic behavior typical of severe overfitting; however, training-set metrics were not explicitly compared against test-set metrics in the notebook, so overfitting cannot be fully ruled out and is noted as a limitation in Section 15.

13. RESULTS AND FINDINGS

1. Holiday weeks show a statistically significant increase in average weekly sales compared to non-holiday weeks ($t = 2.680$, $p = 0.0076$; Section 9, Figure 2), confirming that holiday-related demand materially moves revenue and should be planned for explicitly.
2. Store identity is the strongest driver of sales variation: average weekly sales range roughly eight-fold between the top and bottom stores (Figure 3), far exceeding the influence of any single macroeconomic indicator, indicating that structural differences between stores (size, location, format) matter more than short-term economic fluctuations.
3. Macroeconomic indicators (CPI, Unemployment, Fuel_Price, Temperature) show comparatively weak individual linear correlation with Weekly_Sales (all $|r| < 0.11$; Figure 4), but tree-based ensembles capture non-linear interactions among them and vastly outperform Linear Regression (R^2 0.96–0.97 vs 0.15).
4. Hyperparameter tuning meaningfully improved Gradient Boosting (R^2 from 0.917 to 0.974) while leaving Random Forest essentially unchanged, and the tuned Gradient Boosting model is the best-performing model overall (Table 7, Figure 6), validated via 5-fold cross-validation during the search.
5. Average sales rise through the second half of the year and peak in November and December (Figure 1), reflecting year-end holiday shopping seasonality that any deployed forecasting model should account for.

14. DISCUSSION

The results indicate that weekly sales forecasting for this retailer is fundamentally a store-level problem overlaid with seasonal and holiday effects, rather than one primarily explained by regional macroeconomic swings. The poor performance of Linear Regression compared to tree-based ensembles reinforces that the relationship between the available predictors and Weekly_Sales is non-linear and involves interactions (for example, how a holiday's effect on sales may differ by store size). The improvement from tuning Gradient Boosting, but not Random Forest, suggests Gradient Boosting was more sensitive to its default hyperparameters (particularly learning rate and tree depth) and benefited more from systematic search. From a decision-making perspective, a practitioner should treat the significant holiday effect and large store-to-store variation as the two most actionable levers, while treating the observed correlations between macroeconomic variables and sales as associative rather than causal, since no experimental manipulation of these variables was performed.

15. LIMITATIONS

- The analysis is limited to the 45 stores and the observation window present in the dataset; results may not generalize to other regions, store formats, or time periods.

- Store-level structural drivers of sales (size, format, local demographics, competitor proximity) were not directly available as features; Store was used only as an identifier, which limits interpretability of why particular stores outperform others.
- Weekly-level (not daily or department-level) aggregation limits the granularity of the forecasts and may mask within-week patterns relevant to staffing.
- Training-set performance was not directly compared against test-set performance for the ensemble models, so the possibility of mild overfitting cannot be fully excluded (Section 12.3).
- The 34 flagged outliers in Weekly_Sales were retained as genuine values based on qualitative judgement rather than a formal robustness check of model sensitivity to them

16. CONCLUSION

This project set out to forecast weekly Walmart store sales using store identity, holiday indicators, and macroeconomic variables. After cleaning the data (which required minimal intervention), engineering calendar features, and exploring the relationships between predictors and Weekly_Sales, four regression models were trained and compared. Linear Regression proved inadequate for this non-linear problem, while tree-based ensembles, especially a hyperparameter-tuned Gradient Boosting model ($RMSE \approx 92,146$, $R^2 \approx 0.974$), achieved strong predictive performance on unseen data. The analysis confirmed that store identity and holiday timing are the dominant drivers of weekly sales, while individual macroeconomic indicators contribute weaker but still useful signal when combined non-linearly. Overall, the project demonstrates that a tuned ensemble regression model can provide a practically useful weekly sales forecast to support retail planning.

17. RECOMMENDATIONS AND FUTURE WORK

17.1 Recommendations

- Inventory & Staffing: Increase stock and staffing ahead of known holiday weeks, particularly for historically high-performing stores, since holiday weeks show a statistically significant sales uplift (Section 9).
- Store-level Strategy: Since store identity strongly affects sales (Section 8.3), develop store-specific rather than one-size-fits-all forecasting and promotional strategies.
- Macro-monitoring: Continue tracking CPI and unemployment trends regionally; although individually weak (Section 8.4), their effect compounds with other factors in ensemble model predictions.
- Model Deployment: Deploy the tuned Gradient Boosting model (Section 12.1) for weekly sales forecasting, retraining periodically (e.g., monthly) as new sales data accumulates.

17.2 Future Work

Future extensions could incorporate additional external data such as regional events, local competitor activity, and detailed promotional calendars to further improve model accuracy; explicitly compare training vs test performance to formally quantify overfitting; add store-level demographic or format features; and validate the model with more rigorous time-series cross-validation, given the sequential nature of weekly sales data.

18. REFERENCES

- IDRA Summer Training Program. (2026). IDRA Summer Training Program lectures and learning materials.
- IDRA Summer Training Program. (2026). Walmart Weekly Sales Dataset (P_6_Walmart.csv). Provided through the IDRA Summer Training Program Learning Management System.

- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- Scikit-learn. (2026). KFold — scikit-learn documentation. KFold documentation
- Scikit-learn. (2026). Tuning the hyper-parameters of an estimator — GridSearchCV. GridSearchCV documentation
- GeeksforGeeks. (2026). Random Forest Algorithm in Machine Learning. Random Forest article
- GeeksforGeeks. (2026). Gradient Descent Algorithm and its Variants. Gradient Descent article
- CampusX. (n.d.). Machine Learning / Data Science lecture playlist. YouTube. CampusX YouTube

APPENDIX A. CODE.

A.1 Data Loading and Inspection

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from scipy import stats

df = pd.read_csv("P_6_Walmart.csv")
df.info(); df.shape; df.head()
df.describe()
df.isna().sum()
df.duplicated().sum()
df["Store"].nunique()
```

A.2 Data Cleaning

```
df["Date"] = pd.to_datetime(df["Date"], format="%d-%m-%Y")

Q1 = df["Weekly_Sales"].quantile(0.25)
Q3 = df["Weekly_Sales"].quantile(0.75)
IQR = Q3 - Q1
lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
outliers = df[(df["Weekly_Sales"] < lower) | (df["Weekly_Sales"] > upper)]
```

A.3 Preprocessing

```
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
df['WeekOfYear'] = df['Date'].dt.isocalendar().week.astype(int)
df['Week'] = df['Date'].dt.isocalendar().week.astype(int)
df['Day'] = df['Date'].dt.day
df['Quarter'] = df['Date'].dt.quarter
df = df.sort_values(['Store', 'Date']).reset_index(drop=True)

FEATURES = ["Store", "Holiday_Flag", "Temperature", "Fuel_Price",
            "CPI", "Unemployment", "Month", "Week", "Quarter"]
TARGET = "Weekly_Sales"
```

A.4 EDA and Visualization

```
store_avg_sales = df.groupby("Store")["Weekly_Sales"].mean().sort_values(ascending=False)
monthly_avg = df.groupby("Month")["Weekly_Sales"].mean()
```

```
corr_matrix = df[["Weekly_Sales", "Temperature", "Fuel_Price",
                  "CPI", "Unemployment", "Holiday_Flag"]].corr()

holiday_sales = df.loc[df["Holiday_Flag"] == 1, "Weekly_Sales"]
non_holiday_sales = df.loc[df["Holiday_Flag"] == 0, "Weekly_Sales"]
t_stat, p_val = stats.ttest_ind(holiday_sales, non_holiday_sales, equal_var=False)
```

A.5 Model Training and Evaluation

```
from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

X_train, X_test, y_train, y_test = train_test_split(
    df[FEATURES], df[TARGET], test_size=0.2, random_state=42)

models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(random_state=42, n_estimators=200),
    "Gradient Boosting": GradientBoostingRegressor(random_state=42),
}
# ... fit / predict / evaluate loop, then RandomizedSearchCV / GridSearchCV tuning
# (see full source notebook, project_6.ipynb, for the complete implementation)
```

APPENDIX B. ADDITIONAL FIGURES AND TABLES

Additional charts produced in the source notebook — the overall weekly sales trend line across all stores, a boxplot of Weekly_Sales by Holiday_Flag, a boxplot of Weekly_Sales distribution across all 45 stores, and scatter plots of Weekly_Sales against Temperature, Fuel_Price, CPI, and Unemployment — can be inserted here as supplementary evidence.

Model	RMSE	MAE	R ²
Linear Regression	522,241.76	433,650.56	0.1534
Decision Tree	130,891.19	75,105.49	0.9468
Random Forest (baseline)	113,537.68	62,078.42	0.9600
Random Forest (tuned)	113,598.52	61,846.10	0.9599
Gradient Boosting (baseline)	163,769.39	121,585.18	0.9167
Gradient Boosting (tuned)	92,146.03	56,969.44	0.9736

Table 9. Complete Model Performance Summary (All Models, Test Set)

- The scatter plots were used to examine whether **Weekly Sales** have a clear relationship with **Temperature, Fuel Price, CPI, and Unemployment**.
 - **Temperature vs Weekly Sales:** The points are widely scattered, with no clear upward or downward trend. This suggests that temperature has **little linear relationship** with weekly sales.
 - **Fuel Price vs Weekly Sales:** Weekly sales remain spread across different fuel-price levels, with no strong visible trend. Therefore, **fuel price does not appear to have a strong direct relationship** with weekly sales.
 - **CPI vs Weekly Sales:** The data forms several clusters at different CPI values. Although some variation in sales can be observed, there is **no consistent linear relationship** between CPI and weekly sales.
 - **Unemployment vs Weekly Sales:** The points are also widely distributed. There may be a **slight negative relationship**, but it is not strong enough to indicate a clear linear pattern.

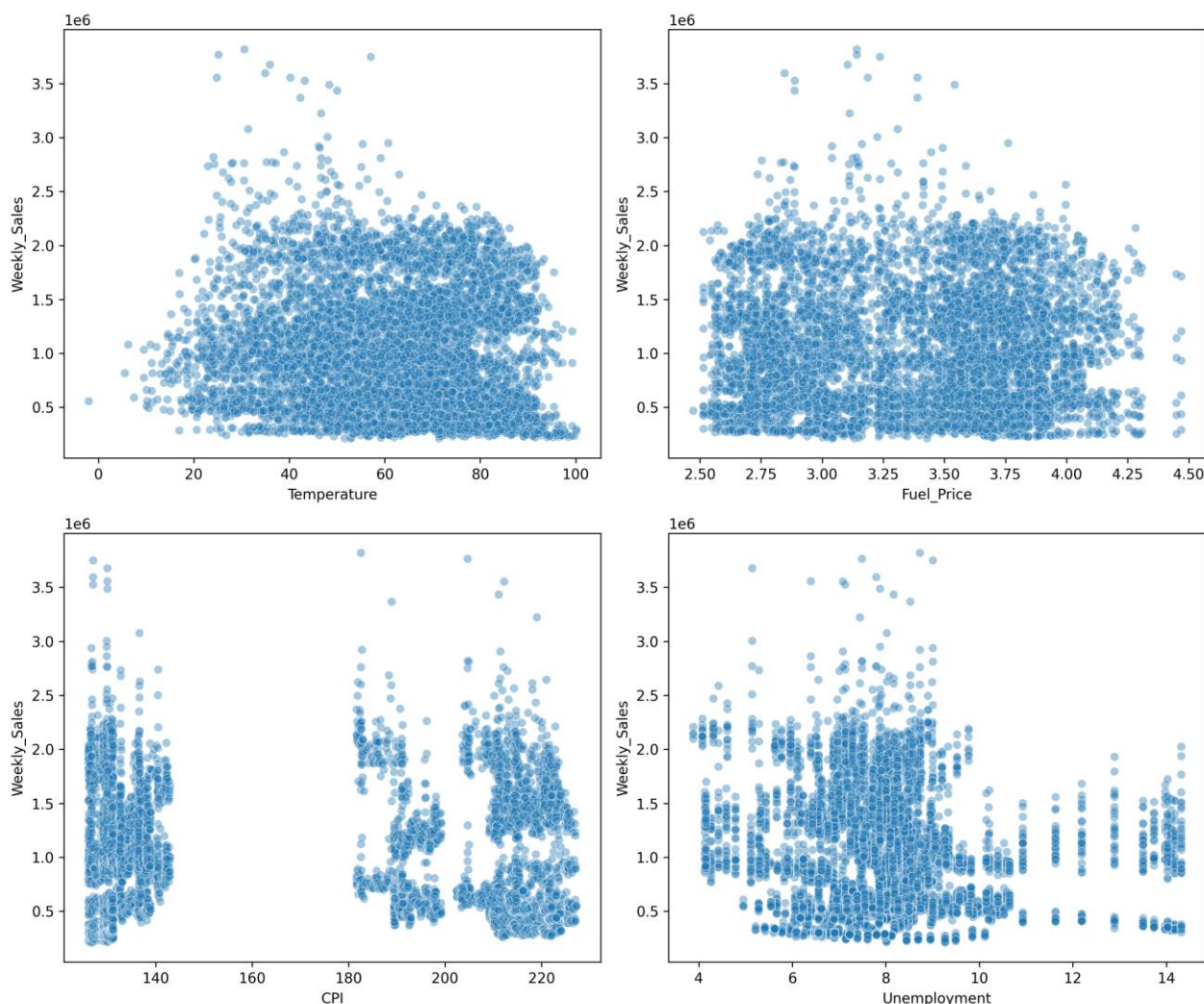


Table 10. Weekly Sales have a clear relationship with Temperature, Fuel Price, CPI, and Unemployment

